

Separating 0–1 Knapsack Inequalities

Computational Integer Optimization · SoSe 2026

Gennesaret Tjusila

The 0–1 knapsack set

Given the set

$$\mathcal{X} = \left\{ x \in \{0, 1\}^n : \sum_{j=1}^n a_j x_j \leq b \right\}.$$

We assume

$$a_j \geq 0 \quad \text{for all } j,$$

complementing if needed with $\bar{x}_j = 1 - x_j$.

A set $C \subseteq N$ is a **cover** if

$$\sum_{j \in C} a_j > b.$$

A cover is **minimal** if $C \setminus \{j\}$ is not a cover for any $j \in C$.

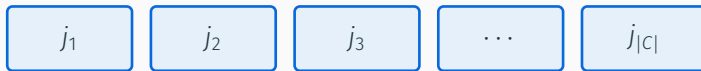
If $C \subseteq N$ is a cover for $\mathcal{X}$, the cover inequality

$$\sum_{j \in C} x_j \leq |C| - 1$$

is valid for $\mathcal{X}$.

Why is the cover inequality valid?

the items in the cover C



Taking every item in C would give

$$\sum_{j \in C} a_j > b.$$

Hence at least one item in C must not be taken.

Strengthening the cover inequality

If C is a cover for $\mathcal{X}$, let

$$E(C) = C \cup \{j : a_j \geq a_i \text{ for all } i \in C\}.$$

Then the strengthened inequality

$$\sum_{j \in E(C)} x_j \leq |C| - 1$$

is valid for $\mathcal{X}$.

Example

Consider the set

$$\mathcal{X} = \{x \in \{0, 1\}^7 : 11x_1 + 6x_2 + 6x_3 + 5x_4 + 5x_5 + 4x_6 + x_7 \leq 19\}.$$

The set $C = \{3, 4, 5, 6\}$ is a valid cover, resulting in the cut

$$x_3 + x_4 + x_5 + x_6 \leq 3.$$

This cut can be strengthened to

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 \leq 3.$$

Lifting the cover inequality

Suppose $x_1 = x_2 = x_7 = 0$. The knapsack inequality becomes

$$6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 19,$$

and

$$x_3 + x_4 + x_5 + x_6 \leq 3$$

remains valid.

We now ask: for which value $\alpha_1 \geq 0$ is

$$\alpha_1 x_1 + x_3 + x_4 + x_5 + x_6 \leq 3 \tag{*}$$

valid when $x_2 = x_7 = 0$?

Lifting x_1

If $x_1 = 0$, inequality $(*)$ is valid for all α_1 .

If $x_1 = 1$, the inequality defining $\mathcal{X}$ becomes

$$6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 8,$$

and α_1 must be such that

$$\alpha_1 + x_3 + x_4 + x_5 + x_6 \leq 3$$

for all $x \in \{0, 1\}^4$ satisfying this knapsack inequality.

The lifting subproblem for x_1

This can be rewritten as

$$\alpha_1 + \underbrace{\max \{x_3 + x_4 + x_5 + x_6 : 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 8, x \in \{0, 1\}^4\}}_P \leq 3.$$

Solving P gives an optimum value of 1, hence

$$\alpha_1 \leq 2.$$

Therefore, the following inequality is valid:

$$2x_1 + x_3 + x_4 + x_5 + x_6 \leq 3.$$

For x_2 , a similar analysis gives

$$\alpha_2 \leq 3 - \max \{2x_1 + x_3 + x_4 + x_5 + x_6 : 11x_1 + 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 13, x \in \{0, 1\}^5\}.$$

This yields $\alpha_2 \leq 3 - 2 = 1$.

A similar calculation gives $\alpha_7 = 0$. Hence we obtain

$$2x_1 + x_2 + x_3 + x_4 + x_5 + x_6 \leq 3.$$

Separating cover inequalities

Given a fractional LP solution x^* , we want to check whether x^* satisfies all cover inequalities.

The cover inequality can be rewritten as

$$\sum_{j \in C} (1 - x_j) \geq 1.$$

Hence it is sufficient to ask whether there is a set $C \subseteq N$ with

$$\sum_{j \in C} a_j > b$$

for which

$$\sum_{j \in C} (1 - x_j^*) < 1.$$

Such a cut is violated by x^* .

Modelling separation as an IP

Let $z_j = 1$ if $j \in C$, and $z_j = 0$ otherwise. We get

$$\begin{aligned} \min \quad & \sum_j \underbrace{(1 - x_j^*)}_{c_j} z_j \\ \text{s.t.} \quad & \sum_j a_j z_j > b, \\ & z_j \in \{0, 1\}. \end{aligned}$$

Since the data are integral, the cover constraint can be written as

$$\sum_j a_j z_j \geq b + 1.$$

For each variable, there is a cost $c_j \geq 0$ given by

$$c_j = 1 - x_j^*.$$

We want to find a set of variables of minimum cost that forms a cover. One can solve this greedily by picking items that minimize the cost paid per unit of weight.

This is again a knapsack problem.

Standard knapsack dynamic programming

Standard setup: weights w_i , profits p_i , and capacity C . The task is to find a set of items J such that

$$\sum_{i \in J} w_i \leq C$$

of maximum profit.

Let $DP[i, w]$ be the maximum profit where only the first i items are considered and the capacity is w .

The standard DP recurrence

$$\text{DP}[i, w] = \begin{cases} \max\{\text{DP}[i-1, w], p_i + \text{DP}[i-1, w - w_i]\}, & w_i \leq w, \\ \text{DP}[i-1, w], & \text{otherwise.} \end{cases}$$

The base case is

$$\text{DP}[0, w] = 0 \quad \text{for all } w.$$

- If we get a new item, we do not need to recompute everything.
- We only need to store the last row.

Inverted dynamic programming

Let $M[i, p]$ be the minimum weight it takes to obtain profit p , where only the first i items are considered.

Then

$$M[i, p] = \min \{M[i - 1, p], w_i + M[i - 1, \max\{0, p - p_i\}]\}.$$

The base cases are

$$M[0, 0] = 0, \quad M[0, p] = +\infty \quad \text{for } p > 0.$$

The answer is given by

$$\max\{p : M[n, p] \leq b\}.$$

Using the inverted DP for lifting

For x_1 , solve

$$\max \{x_3 + x_4 + x_5 + x_6 : 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 8, x \in \{0, 1\}^4\}.$$

For x_2 , solve

$$\max \{2x_1 + x_3 + x_4 + x_5 + x_6 : 11x_1 + 6x_3 + 5x_4 + 5x_5 + 4x_6 \leq 13, x \in \{0, 1\}^5\}.$$

For x_7 , use the already lifted coefficients and capacity 18.

One DP state for the non-cover items

For the non-cover items, use

$$M[0, p] = \{\text{minimum weight needed to obtain profit } p \\ \text{using zero non-cover items}\}.$$

As each non-cover item is lifted, update the last row instead of recomputing the entire dynamic program.